

VGT Project WAARSCHUW

Anomalie detectie in water sensoren met ML.NET

Documentatie Yassine Ouaga

Wat is ML.NET?

ML.NET is een krachtig en cross platform machine learning framework ontwikkeld door Microsoft je kunt er machine learning modellen mee bouwen, trainen en toepassen in C#

project overzicht: Detectie van abnormale watermetingen

In dit project bouw ik een console applicatie in .NET die:

- Metingen van watersensoren uit een csv bestand inleest
- Per sensor bekijkt of er spikes of uitbijters zijn
- De resultaten overzichtelijk in de console toon

Stap 1 – De omgeving instellen

Vereisten:

- .NET 9
- NuGet packages:

```
package Microsoft.ML
Microsoft.ML.TimeSeries
CsvHelper
```

Stap 2 – Datamodellen definiëren

WaterReading.cs

```
public class WaterReading
{
    2 references
    public string Tags { get; set; }
    3 references
    public DateTime Time { get; set; }
    2 references
    public double Distance { get; set; }
}
```

Elke rij uit de csv (sensor naam, timestamp, afstand) en wordt gebruikt om ruwe data uit het CSV-bestand in te lezen.

AnomalyPrediction.cs

```
2 references
public class AnomalyPrediction
{
    [VectorType(3)]
    2 references
    public double[] Prediction { get; set; }
}
```

Output van DetectSpikeBySsa. De Prediction array bevat 3 waarden:

- [0] = 1 als het een spike is, anders 0
- [1] = score van het model
- [2]: p-waarde (hoe sterk het bewijs is dat dit een anomaly is)

SensorData.cs

```
public class SensorData
{
    2 references
    public DateTime Time { get; set; }
    3 references
    public float Value { get; set; }
}
```

Deze wordt gebruikt om de input klaar te maken voor ML.NET maar alleen de waarden (distance) worden geanalyseerd

Program.cs

1. csv inladen

```
string inputFile = "data/wateralarm-vives.csv";
var allReadings = LoadCsv(inputFile);
```

Laadt de csv in en converteert elke rij naar een WaterReading

2. data groeperen op sensor

```
var grouped = allReadings
    .GroupBy(r => r.Tags)
    .ToDictionary(g => g.Key, g => g.OrderBy(r => r.Time).ToList());
```

Zorgt dat elke sensor apart wordt geanalyseerd.

3. ML.NET voorbereiden

```
var mlContext = new MLContext();
```

Maakt een nieuwe ML.NET context aan, wat nodig is voor alle ML.NET bewerkingen : laden van data, trainen van modellen en voorspellingen uitvoeren enzovoort.

4. Detectie van spikes per sensor

We analyseren de gegevens afzonderlijk per sensor id

```
foreach (var (sensor, readings) in grouped)
{
```

doorloopt elk groepje metingen per sensor voor elke groep detecteer je afzonderlijk de anomalies

4.1 input voorbereiden voor ML.NET

```
var sensorData = readings.Select(r => new SensorData
{
    Time = r.Time,
    Value = (float)r.Distance
}).ToList();

var dataView = mlContext.Data.LoadFromEnumerable(sensorData);
```

We zetten de lijst van WaterReading objecten om naar SensorData, die enkel de waarde bevat die we analyseren (Value) samen met de timestamp.

4.2 omzetten naar IDataView

```
var dataView = mlContext.Data.LoadFromEnumerable(sensorData);
```

ML.NET werkt intern met IDataView als datastructuur voor trainingen en voorspellingen

Hier zetten ik de lijst om naar een formaat dat ML.NET begrijpt

4.3 Model bouwen met DetectSpikeBySsa

```
var pipeline = mlContext.Transforms.DetectSpikeBySsa(
    outputColumnName: nameof(AnomalyPrediction.Prediction),
    inputColumnName: nameof(SensorData.Value),
    confidence: 95.0,
    pvalueHistoryLength: sensorData.Count / 2,
    trainingWindowSize: sensorData.Count,
    seasonalityWindowSize: sensorData.Count / 4
);
```

- outputColumnName: De naam van de kolom waarin het model de voorspellingen (1 = anomalie, 0 = normaal) ga plaatsen
- inputColumnName: De input waarde waarop het model werkt. In dit geval de value van de sensor.
- confidence: Het betrouwbaarheids niveau in procent 95 % (95.0 betekent een fout marge van 5%).
- pvalueHistoryLength: Hoeveel eerdere punten gebruikt worden om de p-waarde van het huidige punt te berekenen. Hier: de helft van de dataset.
- trainingWindowSize: Aantal punten dat gebruikt wordt om het patroon te leren. Hier: de volledige dataset.
- seasonalityWindowSize: Hoeveel punten worden vergeleken om periodieke patronen te ontdekken.

maar DetectSpikeBySsa moet weten hoe lang zo'n patroon ongeveer duurt (bv. dag/nacht, week...) en als jij geen idee hebt van de cyclus dan is count 4 zoals in dit geval

4.4 model trainen en toepassen

```
var model = pipeline.Fit(dataView);
var transformed = model.Transform(dataView);

var predictions = mlContext.Data.CreateEnumerable<AnomalyPrediction>(
    transformed, reuseRowObject: false).ToList();
```

trainen het model op de data via Fit. daarna voeren we het uit op dezelfde data (Transform) en halen we de voorspellingen op (CreateEnumerable)

4.5 resultaten weergeven

```
for (int i = 0; i < predictions.Count; i++)
{
    bool isAnomaly = predictions[i].Prediction[0] == 1;

    Console.WriteLine($"{sensorData[i].Time:u}\t{sensorData[i].Value}\t{(isAnomaly ? "YES" : "")}");
}
```

Het script werkt correct: het leest de data in, verwerkt de metingen per sensor en detecteert mogelijke anomalieën met ML.NET (DetectSpikeBySsa).

Toch zijn er beperkingen:

- Niet alle echte uitbijters bij sensor A840415930000001 worden gedetecteerd.
- Bij sensor A8404142B1000003 worden fluctuaties ten onrechte als anomalieën gemarkeerd, terwijl het waarschijnlijk gaat om een normaal circadiaans patroon.

Dit komt deels door de beperkte tijd en ons nog groeiende begrip van wat een 'anomalie' precies betekent in deze context. De gekozen parameters zijn gebaseerd op vuistregels en algemene documentatie, niet op diepgaande domeinkennis of uitgebreide data analyse.

Z-score

documentatie Iban Blondelle

Ik heb geprobeerd om Z-score te implementeren.

De Z-score is een statistische methode die vaak gebruikt wordt om anomalieën (of “outliers”) in data te detecteren. In de zin van waterstandmetingen zou het kunnen automatisch outliers opsporen.

De Z-score geeft aan hoeveel standaardafwijkingen een specifieke meting afwijkt van het gemiddelde van alle metingen.

Dit is de formule:

$$Z = \frac{X - \mu}{\sigma}$$

where:

- X is the data point,
- μ is the mean, and
- σ is the standard deviation.

Een Z-score van 0 betekent dat de meting exact gelijk is aan het gemiddelde. Een positieve Z-score wijst op een meting boven het gemiddelde, een negatieve Z-score op een meting eronder. Hoe verder de Z-score van nul ligt (bijvoorbeeld +3 of -3), hoe groter de afwijking.

Het voordeel van deze methode is dat ze relatief eenvoudig en snel toepasbaar is, zeker als je werkt met één sensor tegelijk.

Het nadeel is dat de methode gevoelig is voor de verdeling van de data: als de data niet normaal verdeeld is, of als er natuurlijke seizoensinvloeden of sprongen voorkomen (zoals bij het manueel bijvullen van een regenput), dan kunnen ook correcte waarden ten onrechte als outlier worden gemarkeerd.

Kort samengevat: Z-score is een nuttig hulpmiddel om systematisch afwijkende metingen in kaart te brengen, maar moet verstandig geïnterpreteerd worden binnen de context van de fysieke processen achter de data.

Een uitbreiding voor dit is Rolling Z-score.

Een rolling Z-score is een uitbreiding van de gewone Z-score die bijzonder nuttig is voor tijdreeksen, zoals in de waterstandmetingen die in de tijd variëren. In plaats van het gemiddelde en de standaardafwijking te berekenen over de volledige dataset, doet de rolling Z-score dit over een schuivend venster (een "rolling window") van beperkte grootte.

De rolling Z-score past zich voortdurend aan lokale omstandigheden aan en is daardoor robuuster tegen:

- Langetermijntrends (bv. seizoensverloop),
- Structurele verschuivingen (bv. onderhoud aan een sensor),
- Variabele meetomstandigheden.

Dit "rollende venster" schuift telkens één stap vooruit: de oudste meting valt eruit, de nieuwste komt erbij. Zo blijft de berekening altijd gebaseerd op de meest recente data. Hiermee detecteer je outliers die binnen de lokale context uitzonderlijk zijn, in plaats van ten opzichte van het globale gemiddelde. Dit helpt om plotselinge spikes of dips te detecteren die buiten het normale patroon vallen, zonder valse alarmen bij normale metingen.

Dit heb ik geprobeerd toe te passen in mijn code. Het werkt enigszins wel, maar hij haalt sommige anomalies niet op die dicht bij elkaar liggen.

Hij haalt ook nog bepaalde resultaten op die mogelijk een anomaly kunnen zijn, maar eigenlijk zijn dit gewoon normale metingen.

Tijl: IQR en gemiddelden vergelijken

Eerst heb ik samen met de rest de github met branches en dit bestand aangemaakt, dan begon ik apart met de csv-data in Excel te openen om grafieken te genereren, om overzichtelijker te kunnen werken/denken.

Grafieken aanmaken was verrassend moeilijk om zonder problemen te doen, toch zeker met de data die ik wou kunnen zien. Uiteindelijk (ook op niet de simpelste manier) heb ik 4 grafieken aangemaakt: één voor elke sensor, zo kan ik enigszins gemakkelijk waarden checken en zien waar uitschieters zich bevinden.

Sommige waarden werden niet meer juist weergegeven, dus ook nog een andere nieuwe Excel geopend met de data, en daarin ook een grafiek met alle 4 de sensorwaarden, elk op een eigen lijn weergegeven.

Dan heb ik met hulp van de grafiek en tabel geprobeerd om berekeningen te bedenken die zouden helpen om uitschieters te vinden, maar vooral ook mijn persoonlijk primair doel: proberen zorgen dat bij sensor 1 de 3 uitschieters die naast elkaar staan alledrie juist worden opgemerkt én dat de twee waarden waar ze naast staan, die net na een grote maar volledig bedoelde sprong in waarde komen, niet worden gezien als uitschieters.

Na veel denken had ik niet meteen een concrete bewerking bedacht, dus besloot ik om voorgestelde manieren uit te proberen, dus haalde ik een deel van de code die al werd aangemaakt door Iban op in mijn eigen branch om niet alleen de data uit het bestand op te halen in het project, maar het ook per sensor te verdelen en chronologisch te ordenen, wat heel handig is.

Oorspronkelijk was ik van plan om met Z-score te proberen werken, zo mogelijk samen met nog een andere methode (voorgesteld of zelf bedacht), dus bekeek ik de website die ons werd gegeven waarin er uitleg stond over Z-score, met standaardformule en voorbeeldcode in Python. Verder op de pagina stonden er ook andere methodes zoals IQR, wederom met standaardformule en Python-voorbeeldcode.

Omdat: 1. Iban Z-score zou proberen, 2. Yassine er zelfs al eens succesvol mee heeft gewerkt, 3. er als nadeel aan Z-score werd gegeven dat het niet ideaal zou zijn voor asymmetrische data en dat het sensitief zou zijn voor extreme uitschieters, 4. er over IQR verteld werd dat het niet zo goed zou werken voor kleine datasets, maar dat het wel goed om kan met asymmetrische datasets en extreme uitschieters, 5. ik beide methodes vanuit Python zou moeten interpreteren, de code van IQR korter was, en ik ook beter begreep wat er werd berekend voor IQR - besloot ik om IQR te proberen toepassen in plaats van Z-score.

Voor het omzetten van de Python-code naar C# heb ik hulp gevraagd aan ChatGPT, omdat ik nog nooit met Python heb gewerkt en er functies werden gebruikt die in C# niet bestaan. Zo voegde ik een apart bestand toe dat de functie bevatte om met kwartielen en percentielen te werken, waar ik uit het gewoonlijke bestand dan aan kon om die functie toe te passen. Dan was het gewoon nog de formule voor de IQR opstellen en zorgen dat er bij het overlopen van de data werd aangeduid in de console welke waarden er door de IQR-methode als verdacht werden beschouwd - maar voorlopig nog als "mogelijke anomalie," omdat IQR niet nauwkeurig genoeg was en ik sowieso nog steeds een eigen

bewerking probeerde te verzinnen om het nauwkeuriger te laten werken of toch op zijn minst een dubbelcheck te hebben.

Ik probeerde zoals Wim eens had vermeld de uitschieters te herkennen a.d.h.v. vergelijkingen met gemiddelden van de vorige en volgende waarden, maar ik merkte te weinig verschil tussen de gemiddelden die ik had laten berekenen om een onderscheid te maken tussen uitschieters en normale waarden - zeker bij de 5 waarden waar ik het meest mee inzat (3 opeenvolgende uitschieters vlak na 2 normale waarden na een grote sprong).

Ik verbreedde dus mijn dataset, wat zowel voor- als nadelen had. Om zeker te zijn dat de drie uitschieters niet genoeg invloed zouden hebben op een gemiddelde om ze normaal te doen lijken, liet ik voor elke waarde het gemiddelde van de vorige 9 waarden en het gemiddelde van de volgende 9 waarden berekenen, en zouden alleen waarden die verdacht veel afweken van de gemiddelden aan beide kanten gezien worden als uitschieters. Zo was ik relatief zeker dat geldige waarden zowel voor als na grote sprongen in waarde niet foutief zouden worden verwijderd. Het nadeel is natuurlijk dat de eerste en laatste 9 waarden van elke sensor hier niet op konden getest worden. Dit lijkt te werken naar behoren. We hadden ook een probleem met gitignore, maar na een beetje opzoekwerk was dat wel opgelost.

Nadat we merkten dat we vergeten waren dat er ook werd gevraagd om te kunnen aanduiden waar er grote stijgingen of dalingen waren, schoot ik in actie aangezien het de laatste dag was, met het idee dat ik misschien kon zorgen dat er bij elke niet-anomalie werd gecheckt of die een gemiddelde van 9 waarden voor of achter zich had die veel meer verschilde van de huidige waarde dan het andere gemiddelde, en of de vorige (geldige) waarde het omgekeerde had. Zo zou ik bijvoorbeeld kunnen zeggen dat als het ene punt een veel lagere waarde heeft dan de gemiddelde waarde van 9 punten erna (maar niet veel verschilt van het gemiddelde ervoor), en het eerstvolgende punt een veel hogere waarde heeft dan de gemiddelde waarde van 9 punten ervoor (maar niet veel verschilt van het gemiddelde erna), dat daar een sprong gebeurde in het waterniveau.

Helaas bleek dit moeilijker dan gedacht, en heb ik het deels-werkend moeten neerleggen. Om te zeggen of een waarde speciaal hoog of laag is, maakte ik deels gebruik van de IQR, maar dit was blijkbaar niet ideaal. Bij sensor 1 werkt het nog het best van allemaal, want daar lukt het om 3 sprongen correct te herkennen. Spijtig genoeg lukte het niet om de vierde, kleinere sprong te vinden. Bij de andere sensoren was het alleen maar minder goed, want er werd namelijk geen enkele sprong meer gevonden, omdat de IQR te groot was. Ik probeerde een kleinere waarde of percentage van de IQR, maar toen begon het zich sprongen in te beelden bij sensor 1, waar er helemaal geen waren. Door een gebrek aan tijd heb ik dit niet verder kunnen onderzoeken.